МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №3
по курсу «Программирование графических процессоров»

Классификация и кластеризация изображений на GPU.

Выполнил: Федорова Екатерина Васильевна
Группа: М8О-408Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы: научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки потоков.

Вариант: 3, метод наименьшего расстояния. Входные данные: на первой строке задается путь к исходному изображению, на второй — путь к конечному изображению. На следующей строке число nc — количество классов. Далее идут nc строчек, описывающих каждый класс. В начале j-ой строки задается число npj — количество пикселей в выборке, за ним следуют npj пар чисел — координаты пикселей выборки. $nc \le 32; np_j \le 2^{19}; w * h \le 4 * 10^8$.

Оценка вектора средних:

$avg_j = \frac{1}{np_j} \sum_{i=1}^{np_j} ps_i^j$, где ps — i-ый пиксель из j-ой выборки.

Необходимо реализовать метод минимального расстояния. Исходное изображение представляет собой “экранный буфер”, на выходе в альфа канале должен быть номер кластера.

Для некого пикселя p номер класса jc определяется формулой:

$$jc = arg\max_j \left[-(p - avg_j)^T * (p - avg_j) \right] = arg\min_j \left[(p - avg_j)^T * (p - avg_j) \right] = arg\min_j \left\| p - avg_j \right\|^2.$$

Программное и аппаратное обеспечение

GPU: NVIDIA GeForce RTX 4050 Laptop GPU
Compute capability: 8.9
Общая память: 6141 MiB, занятая память: 15 MiB, свободная память: 5759 MiB
Графическая память: 5772 MB
Разделяемая память (по умолчанию): 48 KB
Разделяемая память (opt-in, на блок): 99 KB
Разделяемая память (на мультипроцессор): 100 KB
Константная память: 64 KB
Кол-во регистров на блок: 65536
Кол-во регистров на мультипроцессор: 65536
Макс. кол-во потоков на блок: 1024
Макс. размерность блока: 1024 x 1024 x 64
Макс. кол-во потоков на мультипроцессор: 1536
Макс. кол-во блоков на мультипроцессор: 24
Максимальный размер сетки: 2147483647 x 65535 x 65535
Мультипроцессоры: 20

CPU: AMD Ryzen 7 7735HS with Radeon Graphics
Архитектура: x86_64
Address sizes: 48 bits physical, 48 bits virtual

CPU(s): 16
Потоков на ядро: 2
Ядер на сокет: 8
Сокетов: 1
CPU max MHz: 4829,0000
CPU min MHz: 400,0000
Виртуализация: AMD-V
Caches (sum of all):
L1d: 256 KiB (8 instances)
L1i: 256 KiB (8 instances)
L2: 4 MiB (8 instances)
L3: 16 MiB (1 instance)

RAM

	Всего	Занято	Свободно	Общая	Буф/врем.	Доступно
Память	14Gi	7,0Gi	2,3Gi	166Mi	5,9Gi	7,8Gi
Подкачка	2,0Gi	1,2Gi	795Mi			

SSD: DGSM4512GM63T, 476,9G, 0 disk
Файл.система: /dev/nvme0n1p3
Тип: ext4
Размер: 465G
Использовано: 107G

ОС и ПО

Название: Ubuntu 24.04.4 LTS
Версия: 24.04.4 LTS (Noble Numbat)
Ядро: 6.8.0-142-generic
NVIDIA-SMI: 595.84
Driver Version: 595.84
CUDA Version: 13.2
CUDA Toolkit: 12.0 (nvcc V12.0.140)
VSCode: 1.138.0, x64

Метод решения

Для каждого пикселя изображения определяется ближайший по цвету кластер. Сначала для каждого кластера вычисляется средний цвет по каналам R, G и B на основе заданных координат образцов. Оценка средних вычисляется по формуле (дана при постановке задачи):

$avg_j = \frac{1}{np_j} \sum_{i=1}^{np_j} ps_i^j$, где ps — i-ый пиксель из j-ой выборки.

Затем для каждого пикселя вычисляется квадрат евклидова расстояния до среднего цвета каждого класса и выбирается минимальное:

$$jc = arg\max_j \left[-\left(p - avg_j\right)^T * \left(p - avg_j\right) \right] = arg\min_j \left[\left(R_{ps} - avg_j^r\right)^2 + \left(G_{ps} - avg_j^g\right)^2 + \left(B_{ps} - avg_j^b\right)^2 \right],$$

где R_j, G_j, B_j — цветовые компоненты пикселя. Индекс выбранного класса записывается в альфа-канал A пикселя, а исходные значения каналов R, G и B сохраняются без изменений.

В работе реализованы три версии программы:

1. **CUDA без измерения времени** — выполняет классификацию пикселей на графическом процессоре.
2. **CUDA с измерением времени** — выполняет тот же алгоритм и дополнительно измеряет время работы CUDA-ядра с помощью CUDA Events.
3. **CPU** — выполняет классификацию последовательно на центральном процессоре и служит для сравнения производительности.

В CUDA-версиях используется фиксированная конфигурация запуска. Для обработки изображения допустимого размера ядро использует цикл с шагом, равным общему количеству потоков в сетке (grid-stride loop). Это позволяет обработать все пиксели независимо от размера изображения при заданной конфигурации запуска.

Средние цвета классов передаются в CUDA-ядро через константную память (constant memory), предназначенную для хранения небольшого объёма данных, доступных только для чтения. Данные изображения передаются на графический процессор в виде массива пикселей, каждый из которых содержит четыре канала: R, G, B и A.

Для оценки производительности измеряется время непосредственно выполнения CUDA-ядра. Время последовательного алгоритма на CPU измеряется средствами библиотеки std::chrono. В обоих случаях измерение проводится отдельно от чтения и записи файлов, вычисления средних цветов классов и передачи изображения между CPU и GPU, что позволяет сравнить длительность этапа классификации пикселей.

Описание программы

Состав проекта

Файл	Назначение
lab3_cpu.cpp	эталонная CPU-реализация, использующая последовательный цикл и замер времени через `std::chrono`
lab3.cu	простейшая GPU-версия без тайминга для отправки в чекер
lab3_timer.cu	GPU-версия с таймингом через CUDA events, основная для отчёта
tobin.py	скрипт преобразования картинки в необходимый для лабораторной формат бинарного входного файла
topng.py	скрипт преобразования бинарного файла в картинку

Также в проекте присутствуют тестовые файлы. В папке test/images сохранены все варианты картинок и соответствующие бинарные файлы, а также демонстрационные варианты после кластеризации. В папке test/in собраны текстовые файлы: FullHD_4classes_100samples.txt, FullHD_16classes_1000samples.txt, FullHD_32classes_10000samples.txt, 4K_4classes_100samples.txt, 4K_16classes_1000samples.txt, 4K_32classes_10000samples.txt, 8K_4classes_100samples.txt, 8K_16classes_1000samples.txt, 8K_32classes_10000samples.txt — входные данные для быстрого тестирования по времени; FullHD_demo.txt, 4K_demo.txt, 8K_demo.txt — входные данные для тестирования-демонстрации.

Основные типы данных

- uint32_t — хранение ширины и высоты изображения в заголовке входного файла;
- uint64_t — вычисление произведения ширины и высоты изображения для проверки ограничения на количество пикселей без переполнения;
- double — для сумм накопления при подсчёте среднего;
- uchar4 — структура CUDA, содержащая четыре 8-битных компонента пикселя RGBA, используется в CUDA-версиях;
- float3 — структура CUDA для хранения среднего цвета класса по каналам R, G и B;
- std::vector<uchar4> — хранение пикселей в оперативной памяти CPU в CUDA-версиях;
- std::vector<unsigned char> — хранение байтов изображения в оперативной памяти CPU в последовательной версии;
- __constant__ float3 d_avg[32] — массив в памяти констант GPU для хранения средних цветов классов; используется в CUDA-ядре;

- std::chrono::steady_clock — измерение времени выполнения последовательного алгоритма на CPU;
- cudaEvent_t — события CUDA, используемые для измерения времени выполнения ядра на GPU.

Реализованное ядро

В CUDA-версиях программы реализовано единственное ядро midimet. Его сигнатура:

```cpp
__global__ void midimet(
    uchar4* __restrict__ pixels,
    int n, int nc
)```

Параметры ядра:

- pixels: массив пикселей изображения, расположенный в глобальной памяти GPU. После обработки в альфа-канале каждого пикселя хранится индекс назначенного класса.
- n: общее количество пикселей изображения.
- nc: количество классов.

Средние цвета классов хранятся в массиве d_avg в памяти констант GPU и поэтому не передаются ядру отдельным параметром.

Квалификаторы:

- __global__: обозначает CUDA-ядро, которое вызывается с CPU и исполняется на GPU.
- __restrict__: сообщает компилятору, что через указатель pixels осуществляется доступ к области памяти, не перекрывающейся с другими доступными ядру областями через указатели с аналогичными ограничениями. Это позволяет применять дополнительные оптимизации.

Каждый поток вычисляет начальный индекс обрабатываемого пикселя на основе номера блока и номера потока внутри блока. Далее поток последовательно обрабатывает пиксели с шагом, равным общему количеству потоков в сетке CUDA (grid-stride loop). Благодаря этому фиксированная конфигурация запуска позволяет обработать все пиксели изображения допустимого размера.

Для каждого пикселя ядро считывает значения каналов R, G и B, затем перебирает все классы и вычисляет квадрат евклидова расстояния между цветом пикселя и средним цветом каждого класса. Выбирается класс с минимальным расстоянием. Его индекс записывается в альфа-канал пикселя, а остальные три канала сохраняются без изменений. При равенстве расстояний выбирается класс с наименьшим индексом, поскольку обновление результата выполняется только при строгом уменьшении расстояния.
```

Перед запуском ядра изображение копируется из оперативной памяти CPU в глобальную память GPU. Средние цвета классов передаются в память констант с помощью cudaMemcpyToSymbol. После выполнения ядра обработанное изображение копируется обратно в оперативную память и записывается в выходной файл.

В CUDA-версии с измерением производительности используются CUDA Events. События записываются до и после запуска ядра, а затем с помощью cudaEventElapsedTime вычисляется продолжительность выполнения. Измерение не включает чтение и запись файлов, выделение памяти и копирование изображения между CPU и GPU.

CPU-версия

В CPU-версии аналогичные вычисления выполняются непосредственно в функции main() с помощью последовательного цикла по всем пикселям изображения и вложенного цикла по классам.

Сначала для каждого класса вычисляются средние значения каналов R, G и B на основе заданных координат образцов. Для накопления сумм используются переменные типа double, после чего средние значения сохраняются в массиве avg.

Затем программа последовательно обходит все пиксели. Для каждого пикселя вычисляется квадрат расстояния до среднего цвета каждого класса, определяется класс с минимальным расстоянием, а его индекс записывается в четвёртый байт пикселя — альфа-канал A. Исходные значения RGB при этом не изменяются.

Для измерения времени используется std::chrono::steady_clock. Замер охватывает только цикл классификации пикселей, без чтения и записи файлов и вычисления средних цветов классов. Это позволяет сравнить продолжительность вычислений CPU и CUDA, хотя при сравнении также следует учитывать различия в механизмах измерения и накладных расходах выполнения.

Для лучшей визуализации в CPU-версию добавлен вывод в стандартный поток значения R, G, B каналов вектора средних. Также в альфа канал записывался номер кластера, умноженный на коэффициент 35 для контраста.

Все три версии используют одинаковый алгоритм классификации и формируют выходное изображение в одном формате. Это позволяет сравнивать результаты обработки и производительность реализаций.

Вспомогательные функции

Функция checkCuda:
```cpp
bool checkCuda(cudaError_t error, const char* operation);
```

```

Универсальная проверка кода возврата CUDA. При ошибке печатает имя операции и текст ошибки (cudaGetErrorString) в stderr и возвращает false. Позволяет в main строить цепочки проверок без потери читаемости.

Функции formatError:
```cpp
void formatError(const std::string& context, const std::string& err)
```
Универсальный вывод переданной текстовой ошибки. Печатает в stderr контекст ошибки и её саму в формате «ERROR: ...».

Общая структура программ

1. Чтение имён входного и выходного файлов из стандартного ввода.
2. Чтение заголовка входного файла: ширины W и высоты H. Проверка допустимости размеров и общего количества пикселей.
3. Чтение массива пикселей изображения в оперативную память CPU.
4. Чтение количества классов, количества образцов каждого класса и их координат. Проверка допустимых значений координат.
5. Вычисление среднего цвета каждого класса по каналам R, G и B.
6. (для GPU) Выделение памяти на GPU, копирование изображения и передача средних цветов классов в память констант.
7. Запуск вычислений: CUDA-ядра в CUDA-версиях или последовательного цикла в CPU-версии. В версии CUDA с измерением времени дополнительно используются события start и stop.
8. (для GPU) Копирование результата и вычисление затрачиваемого времени.
9. Запись заголовка и обработанного изображения в выходной файл.
10. (для тестирования и отчёта) Вывод размеров изображения, количества классов и времени выполнения в стандартный вывод.
11. Освобождение выделенных ресурсов, включая память GPU и события CUDA в соответствующих версиях.

Отладочный вывод и тестирование

Для проверки используются три начальные картинки расширением Full HD, 4K и 8K. Каждая из них с помощью скрипта на Python преобразовывались в бинарные файлы, которые использовались для вычисления кластеров. На каждом изображении для замера времени тестировались три случая:

1) 4 кластера, для каждого приводятся 100 образцов;
2) 16 кластеров, для каждого приводятся 1000 образцов;
3) 32 кластера, для каждого приводятся 10000 образцов.
```

Для демонстрационных вариантов вручную выделялись кластеры и образцы для них:

- Full HD: 5 кластеров: «Фон чёрный», «Фон синий», «Лампа», «Свет», «Скафандр»;
- 4K: 5 кластеров: «Облака», «Небо», «Горы», «Земля», «Зелень»;
- 8K: 7 кластеров: «Тёмный лес», «Поля», «Голубая река», «Алый», «Зарево», «Солнце», «Птицы».

Для ускорения ввода на вход подавались вспомогательные файлы .txt из папки test/in. Выходные бинарные демо-файлы преобразовывались обратно в картинки с помощью второго Python-скрипта: про каналу прозрачности визуально можно увидеть группы пикселей, отнесённые к различным кластерам. Изображения для тестов времени расположены в папке test/images, для демонстрации — в папке test/dataOut.

Каждый замер производился четыре раза, из которых выбиралось неточное среднее, то есть субъективное представление о средней работе программы.

Результаты

Изображение	Классы + образцы	Кол-во блоков	Кол-во потоков (в блоке)	Время GPU [ms]	Время CPU [ms]
1920x1080 (Full HD)	4+100	1	32	12.3083	192.494
		1	256	1.75718	
		16	256	0.140288	
		64	1024	0.071424	
		256	1024	0.058368	
		1024	1024	0.085824	
	16+1000	1	32	22.4174	487.281
		1	256	4.24621	
		16	256	0.292864	
		64	1024	0.116736	
		256	1024	0.115712	
		1024	1024	0.140192	
	32+10000	1	32	36.011	1004.46
		1	256	7.4455	
		16	256	0.49664	
		64	1024	0.19456	
		256	1024	0.188416	
		1024	1024	0.21504	
3840x2160 (4K)	4+100	1	32	80.2396	548.517
		1	256	10.9251	
		16	256	0.85408	
		64	1024	0.332736	
		256	1024	0.28672	
		1024	1024	0.310272	
	16+1000	1	32	118.72	2006.25
		1	256	18.1115	
		16	256	1.29712	
		64	1024	0.405536	
		256	1024	0.390976	
		1024	1024	0.443296	
	32+10000	1	32	166.949	3948.82
		1	256	31.3037	
		16	256	2.03366	

		64	1024	0.695296	
		256	1024	0.67584	
		1024	1024	0.735136	
7680 × 4320 (8K)	4+100	1	32	321.15	2119.22
		1	256	43.6027	
		16	256	3.42016	
		64	1024	2.0695	
		256	1024	1.53907	
		1024	1024	1.44179	
	16+1000	1	32	474.005	7860.73
		1	256	67.6362	
		16	256	5.14522	
		64	1024	1.73427	
		256	1024	1.52166	
		1024	1024	1.67526	
	32+10000	1	32	665.903	15855.4
		1	256	115.84	
		16	256	8.04045	
		64	1024	2.68672	
		256	1024	2.57942	
		1024	1024	2.71258	

Полученные изображения
1920x1080 (Full HD)



Вывод в консоль:

```
moscow@moscow-Bravo-15-C7VE:~/Рабочий стол/ЛАБЫ ВСЕ/Cuda/lab3$ ./cpu.out < ./test/in/FullHD_demo.txt
W = 1920, H = 1080
Количество кластеров: 5
Time [ms]: 254.489
0: [ 0.875, 2.875, 3.75 ]
1: [ 12.8, 27, 46.4 ]
2: [ 239.75, 223.5, 161.5 ]
3: [ 97.1667, 102.167, 58.5 ]
4: [ 42.6667, 102, 105.167 ]
```

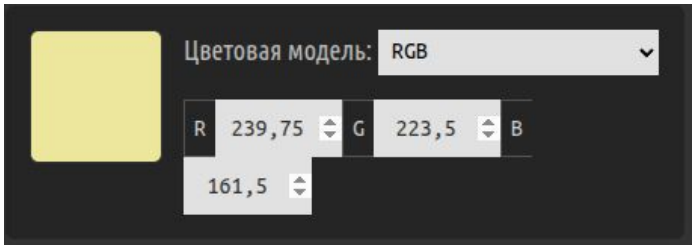
Кластеры:



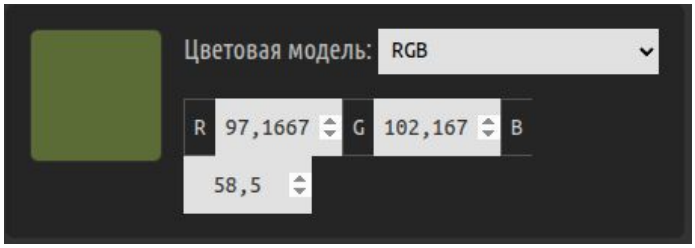
«Фон чёрный»



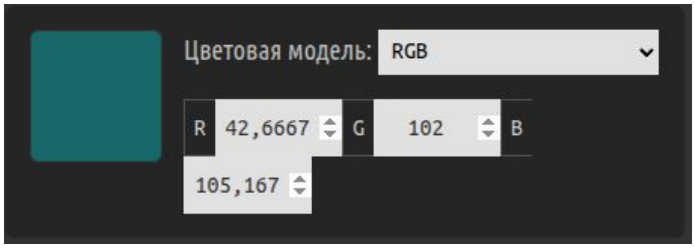
«Фон синий»



«Лампа»

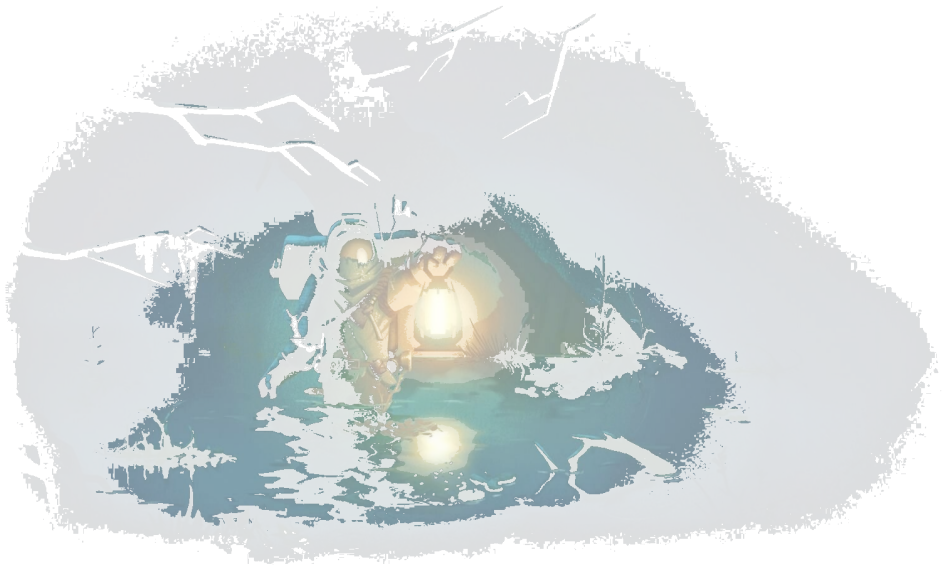


«Скафандр»



«Свет»

Изображение:



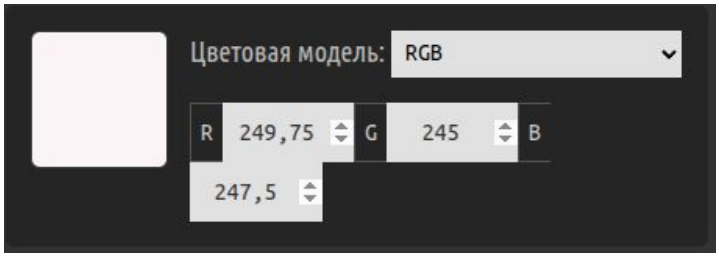
3840x2160 (4K)



Вывод в консоль:

```
moscow@moscow-Bravo-15-C7VE:~/Рабочий стол/ЛАБЫ ВСЕ/Cuda/lab3$ ./cpu.out < ./test/in/4K_demo.txt
W = 3840, H = 2160
Количество кластеров: 5
Time [ms]: 970.071
0: [ 249.75, 245, 247.5 ]
1: [ 161.333, 190, 208 ]
2: [ 62.2857, 109.571, 131.143 ]
3: [ 1.75, 17, 18.75 ]
4: [ 66.3, 100.2, 72.8 ]
```

Кластеры:



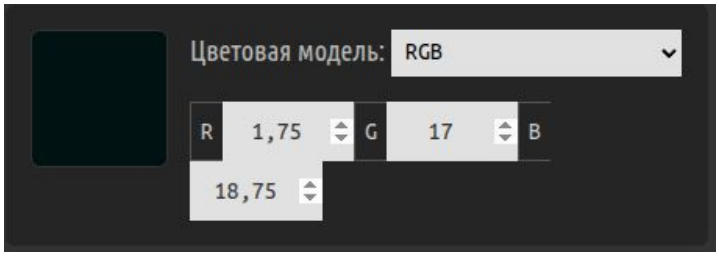
«Облака»



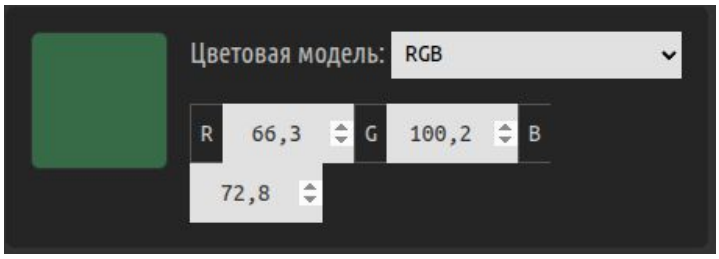
«Небо»



«Горы»



«Земля»



«Зелень»

Изображение:



7680 × 4320 (8K)



Вывод в консоль:

```
moscow@moscow-Bravo-15-C7VE:~/Рабочий стол/ЛАБЫ ВСЕ/Cuda/lab3$ ./cpu.out < ./test/in/8K_demo.txt
W = 7680, H = 4320
Количество кластеров: 7
Time [ms]: 3811.73
0: [ 4.71429, 38.1429, 56 ]
1: [ 53.5556, 105, 132.111 ]
2: [ 63.1667, 166.333, 184.5 ]
3: [ 229.429, 144.286, 150.714 ]
4: [ 252.5, 214.5, 128.25 ]
5: [ 255, 245, 208 ]
6: [ 243.4, 239.2, 238.2 ]
```

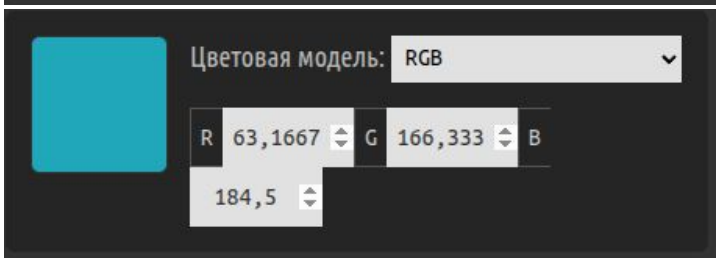
Кластеры:



«Тёмный лес»



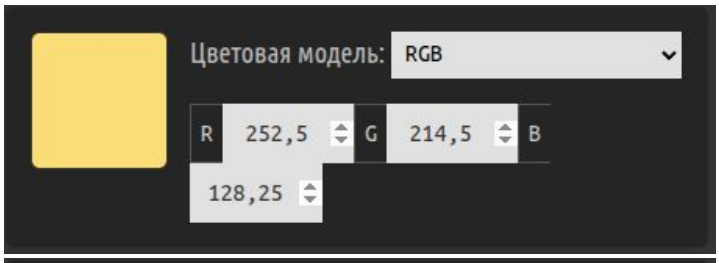
«Поля»



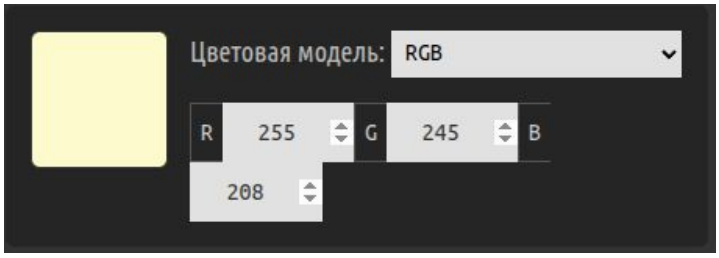
«Голубая река»



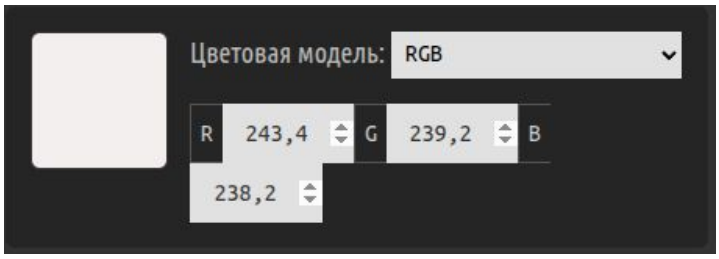
«Алый»



«Зарево»

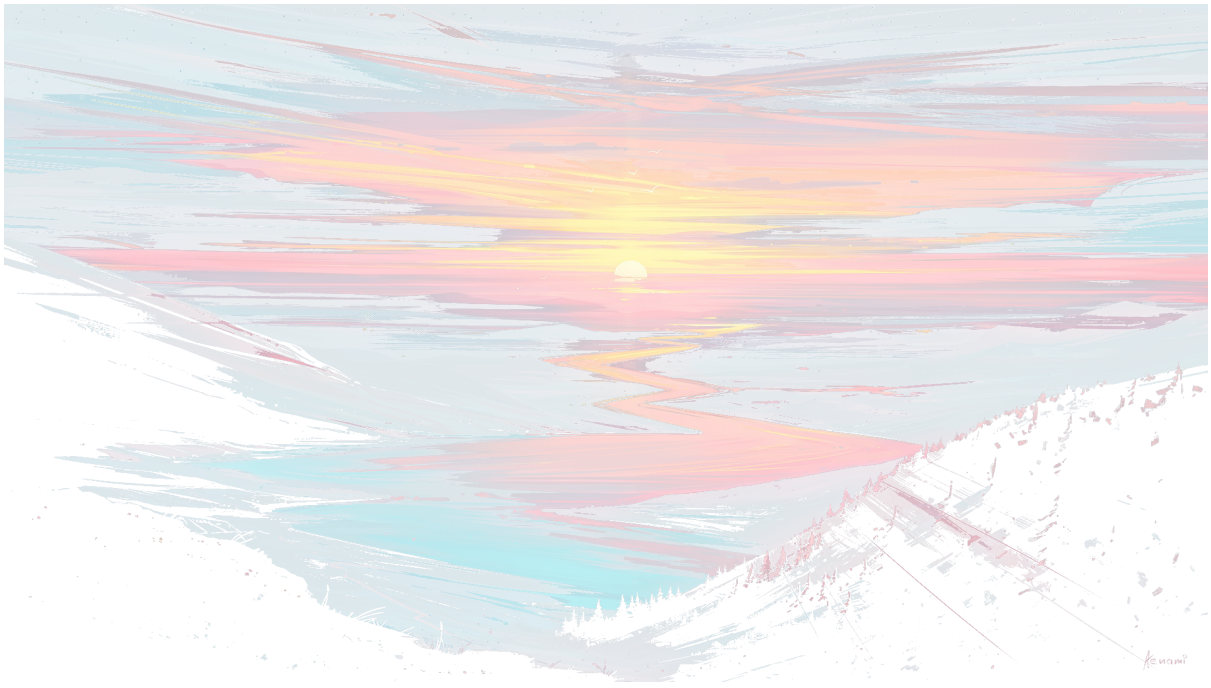


«Солнце»



«Птицы»

Изображение:



Выводы

Область применения

Классификация пикселей по ближайшему среднему цвету класса применяется для сегментации изображений и разделения их на области с похожими цветовыми характеристиками. Для каждого класса по заданным образцам вычисляется средний цвет по каналам R, G и B. Затем каждый пиксель изображения относится к классу, средний цвет которого находится на наименьшем евклидовом расстоянии от цвета этого пикселя. Результат классификации записывается в альфа-канал A, в котором сохраняется индекс назначенного класса, тогда как исходные значения RGB остаются без изменений. Такой подход может использоваться для предварительной сегментации изображений, анализа цветовых областей, обработки графических данных и решения задач кластеризации по цветовому признаку.

Сложность

Обозначим размеры изображения через W и H, количество пикселей — через P=WH, количество кластеров — через nc, а количество образцов для кластера j — через nj.

Временная сложность: $O\left(P*nc+\sum_{j=0}^{nc} nj\right)$. Сначала для каждого класса вычисляется средний цвет по заданным образцам, что требует $O\left(\sum_{j=0}^{nc} nj\right)$ операций. Затем каждый пиксель сравнивается со средними цветами всех классов. На классификацию одного пикселя требуется O(nc) операций, а для всего изображения — O(P*nc). Таким образом, общая сложность определяется количеством пикселей, числом кластеров и количеством образцов.

В последовательной реализации CPU пиксели обрабатываются один за другим. В CUDA-версии классификация распределяется между потоками графического процессора, что позволяет выполнять обработку множества пикселей параллельно. При этом увеличение количества кластеров повышает вычислительную нагрузку, поскольку каждый пиксель необходимо сравнить с большим количеством средних цветов.

Пространственная сложность: O(P+nc) без учёта входных данных и дополнительных служебных структур. Для хранения RGBA-изображения требуется приблизительно 4P байт. Массив средних цветов занимает O(nc) памяти. В CUDA-версии дополнительно выделяется память GPU для изображения и используется память констант для хранения средних цветов классов. Поскольку изображение обрабатывается на месте, отдельный выходной массив на GPU не требуется.

Сравнение результатов

Для оценки производительности использовались изображения Full HD (1920×1080), 4K (3840×2160) и 8K (7680×4320). Для каждого изображения проверялись три набора параметров: 4 класса по 100 образцов, 16 классов по 1000 образцов и 32 класса по 10 000 образцов. В CUDA-версии также исследовалось влияние количества блоков и потоков на время выполнения ядра.

Для изображения **Full HD** время последовательной обработки на CPU составило 192,494 мс для 4 классов, 487,281 мс для 16 классов и 1004,46 мс для 32 классов. Лучшие измеренные времена CUDA составили соответственно 0,058368, 0,115712 и 0,188416 мс. Ускорение относительно CPU составляет приблизительно 3298, 4211 и 5330 раз соответственно.

Для изображения **4K** время CPU составило 548,517 мс для 4 классов, 2006,25 мс для 16 классов и 3948,82 мс для 32 классов. Лучшие времена CUDA составили соответственно 0,28672, 0,390976 и 0,67584 мс. Ускорение относительно CPU составляет приблизительно 1913, 5132 и 5844 раза соответственно.

Для изображения **8K** время последовательной обработки на CPU составило 2119,22 мс для 4 классов, 7860,73 мс для 16 классов и 15855,4 мс для 32 классов. Лучшие времена CUDA составили соответственно 1,44179, 1,52166 и 2,57942 мс. Ускорение относительно CPU составляет приблизительно 1470, 5166 и 6148 раз соответственно.

Во всех рассмотренных случаях CUDA-реализация значительно превосходит последовательную реализацию на CPU по времени выполнения вычислительной части алгоритма. Наибольшее ускорение среди представленных измерений наблюдается для изображения 8K при 32 классах по 10 000 образцов.

Результаты также показывают, что с увеличением количества классов и образцов время выполнения возрастает. Это объясняется увеличением вычислительной нагрузки: каждый пиксель сравнивается со средним цветом каждого класса. В частности, при переходе от 4 к 32 классам растёт количество сравнений, выполняемых для каждого пикселя. Конечно, для большего числа образцов требуется больше операций при вычислении средних цветов, однако их расчёт не входит в замеры времени.

Количество блоков и потоков также существенно влияет на производительность. При запуске с одним блоком время выполнения оказывается значительно выше, чем при использовании нескольких блоков. Однако увеличение количества блоков не всегда улучшает результат: после достижения достаточного уровня параллелизма время может оставаться примерно на одном уровне или увеличиваться. Поэтому оптимальная конфигурация зависит от размера изображения, количества классов и характеристик графического процессора.

Следует учитывать, что время CUDA отражает только выполнение ядра и не включает выделение памяти, передачу данных между CPU и GPU, вычисление средних цветов и операции с файлами. Время CPU также измеряется только на этапе классификации пикселей. Поэтому приведённое ускорение характеризует вычислительную часть

алгоритма, а не полное время выполнения программы. Кроме того, для более надёжного сравнения желательно проводить несколько запусков и вычислять среднее или медианное время.

Демонстрация работы алгоритма

Помимо тестирования производительности, были подготовлены демонстрационные изображения Full HD, 4K и 8K для наглядной проверки работы алгоритма классификации.

В отличие от тестов производительности со случайными координатами, в демонстрационных примерах образцы выбирались целенаправленно, чтобы средние цвета классов соответствовали характерным областям изображения. Это позволяет визуально оценить, как алгоритм разделяет изображение на классы в зависимости от цветового сходства.

В консоль дополнительно выводились вычисленные средние цвета классов в формате RGB. Эти значения использовались для проверки соответствия полученных цветовых характеристик выбранным образцам и для сопоставления с ожидаемыми цветами объектов на изображении.

Для наглядности результата индекс класса при записи в альфа-канал умножался на 35. Таким образом, вместо исходного номера класса j записывалось значение A=35j. Это преобразование применялось исключительно для визуализации: оно увеличивало различия между значениями альфа-канала и делало принадлежность пикселей к разным классам заметнее при просмотре результата. При этом сам алгоритм выбора класса продолжал использовать исходные индексы, а RGB-каналы изображения оставались неизменными.

Демонстрационные изображения показывают, что алгоритм выделяет области со схожими цветовыми характеристиками, однако границы классов не обязательно совпадают с границами реальных объектов. Поскольку классификация основана только на расстоянии между цветом пикселя и средним цветом класса, пиксели разных объектов с похожими цветами могут относиться к одному классу, а внутри одного объекта могут появляться области разных классов. Результат также зависит от выбора образцов и количества классов.

Итог

В ходе лабораторной работы реализован алгоритм классификации пикселей по ближайшему среднему цвету класса в трёх вариантах: CUDA без измерения времени, CUDA с измерением времени выполнения ядра и последовательная CPU-реализация.

Эксперименты показали, что CUDA позволяет существенно сократить время классификации по сравнению с последовательной обработкой на CPU. Наибольшее

измеренное ускорение составило приблизительно 6148 раз для изображения 8K при использовании 32 классов по 10 000 образцов.

Также установлено, что увеличение количества кластеров повышает вычислительную нагрузку, а выбор конфигурации CUDA влияет на производительность. При этом увеличение количества блоков и потоков не гарантирует пропорционального ускорения, поэтому для достижения наилучшего результата необходимо экспериментально подбирать параметры запуска. Полученные результаты подтверждают эффективность параллельной обработки изображений на GPU для задач классификации пикселей.